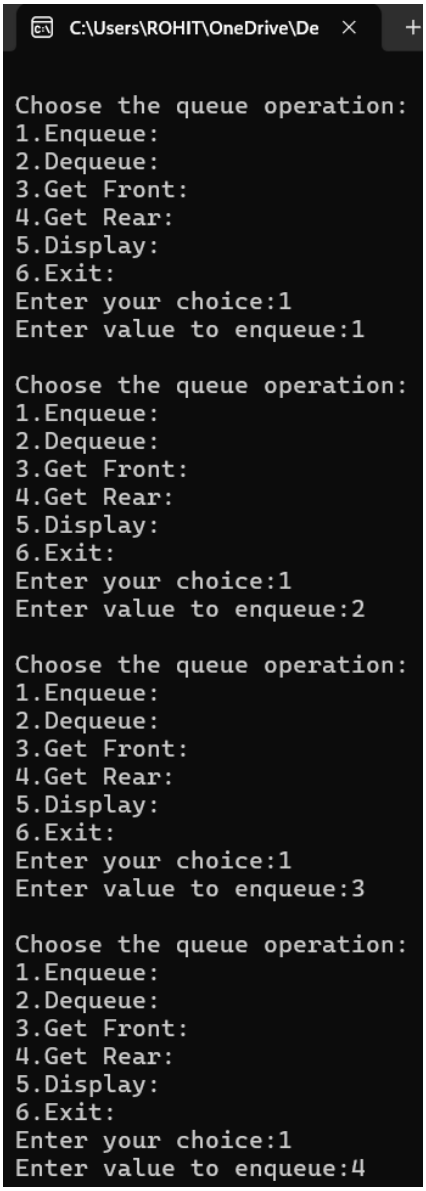
Aim: Implement the following
a. Static implementation of circular queue of integers with following operations: Initialize, insert(), delete (), isempty(), isfull(), display().
b. Dynamic implementation of Queue for integers with the following operations: Insert, Delete, Display, Exit .
c. Dynamic implementation of circular queue of integers with following operations: Initialize, insert(), delete(), isempty(), isfull(), display().
1.Implementation od linear queue using a static array.
CODE:

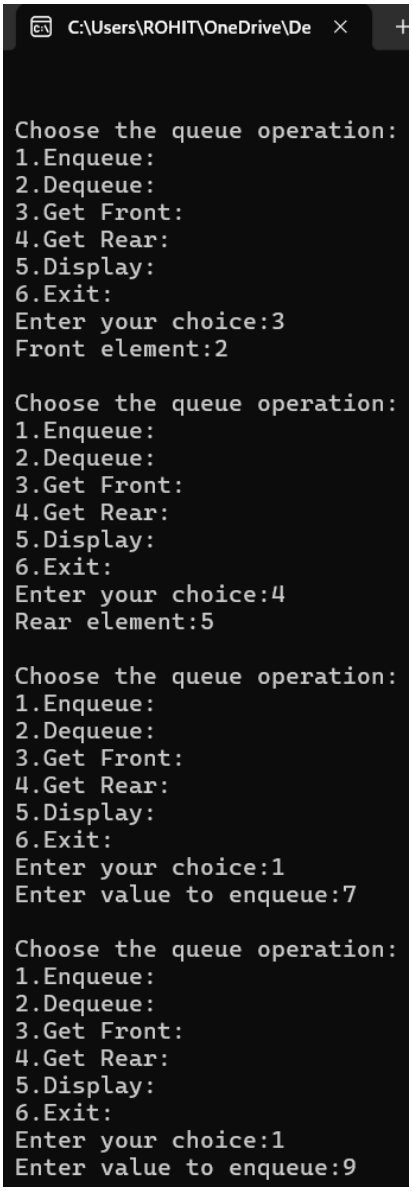
```
#include <iostream>
using namespace std;
class Queue{
	public:
		int a[10];
		int rear, front;
		Queue(){
			rear = front = -1;
		}
		bool isEmpty(){
			return front==-1 || front>rear;
		}
		bool isFull(){
			return rear==9;
		}
		int getFront(){
			if(isEmpty()){
				cout<<"Queue is empty!"<<endl;
				return -1;
			}
			return a[front];
		}
		int getRear(){
			if(isEmpty()){
				cout<<"Queue is empty!"<<endl;
				return -1;
			}
			return a[rear];
		}
		void enqueue(int val){
			if(isFull()){
				cout<<"Queue is full!"<<endl;;
				return;
			}
			if(isEmpty()){
				front = 0;
			}
			rear++;
			a[rear] = val;
```

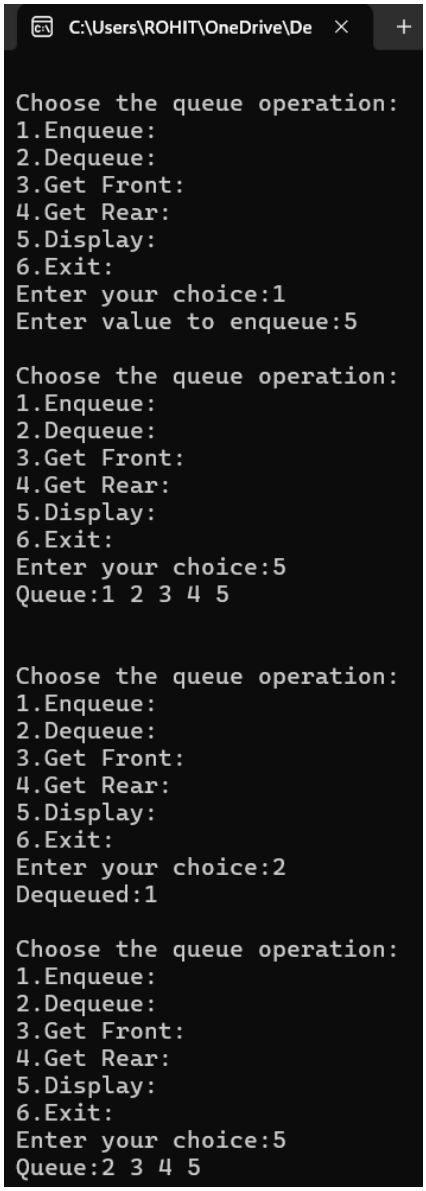
```
        }
        int dequeue(){
            if(isEmpty()){
                cout<<"Queue is Empty!"<<endl;;
                return -1;
            }
            int ans = a[front];
            front++;
            if(isEmpty()){
                front = rear = -1;
            }
            return ans;
        }
        void display(){
            if(isEmpty()){
                cout<<"Queue is empty!";
                return;
            }
            cout<<"Queue:";
            for(int i=front;i<=rear;i++){
                cout<<a[i]<<" ";
            }
            cout<<endl;
        }
    };
int main(){
    Queue q;
  int choice, val;
  while(true){
    cout<<"\nChoose the queue operation:";
    cout<<"\n1.Enqueue:";
    cout<<"\n2.Dequeue:";
    cout<<"\n3.Get Front:";
    cout<<"\n4.Get Rear:";
    cout<<"\n5.Display:";
    cout<<"\n6.Exit:";
    cout<<"\nEnter your choice:";
    cin>>choice;
    switch(choice){
      case 1:
        cout<<"Enter value to enqueue:";
        cin>>val;
        q.enqueue(val);
        break;
      case 2:
        val = q.dequeue();
        if(val != -1){
          cout<<"Dequeued:"<<val<<endl;
```

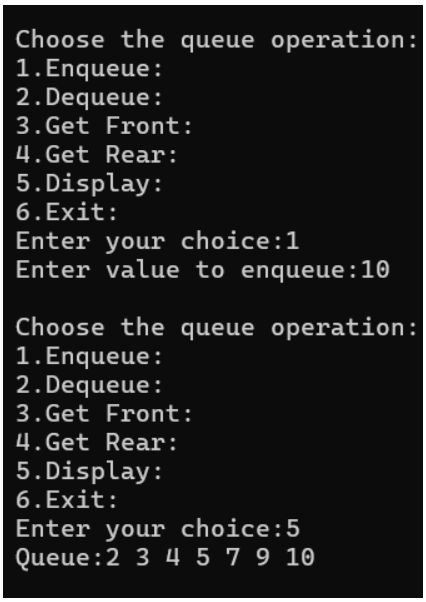
```
            }
            break;
        case 3:
            val = q.getFront();
            if(val != -1){
                cout<<"Front element:"<<val<<endl;
            }
            break;
        case 4:
            val = q.getRear();
            if(val != -1){
                cout<<"Rear element:"<<val<<endl;
            }
            break;
        case 5:
            q.display();
            cout<<endl;
            break;

        case 6:
            cout<<"Exiting..."<<endl;
            return 0;
        default:
            cout<<"Invalid choice!"<<endl;
      }
    }
    return 0;
}
```

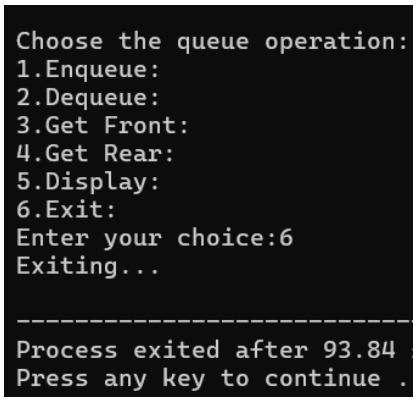
OUTPUT:


C:\Users\ROHIT\OneDrive\De
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:1
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:2
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:3
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:4


C:\Users\ROHIT\OneDrive\De
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:3
Front element:2
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:4
Rear element:5
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:7
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:9


C:\Users\ROHIT\OneDrive\De
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:5
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:5
Queue:1 2 3 4 5
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:2
Dequeued:1
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:5
Queue:2 3 4 5


Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:1
Enter value to enqueue:10
Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:5
Queue:2 3 4 5 7 9 10


Choose the queue operation:
1.Enqueue:
2.Dequeue:
3.Get Front:
4.Get Rear:
5.Display:
6.Exit:
Enter your choice:6
Exiting...
Process exited after 93.84
Press any key to continue .

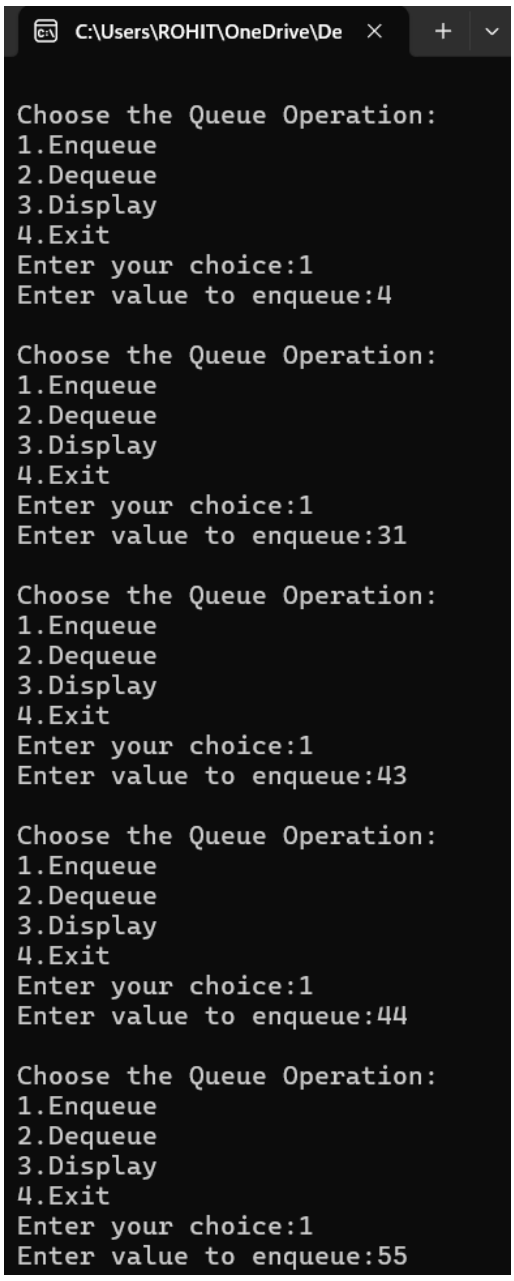
2.Implementation of linear queue using a singly linked list (not circular).

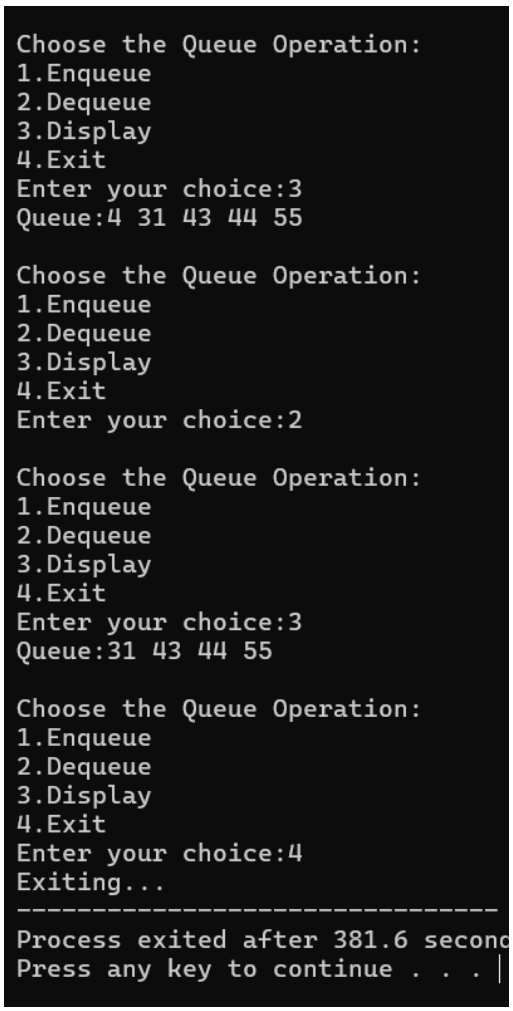
CODE:

```
#include <iostream>
using namespace std;
class Node{
	public:
		int data;
		Node *next;
		Node(int val){
			data = val;
			next = NULL;
		}
	};
class Queue{
	private:
		Node *front, *rear;
	public:
		Queue(){
			front = rear = NULL;
		}
		bool isEmpty(){
			return front == NULL;
		}
		void enqueue(int val){
			Node *t = new Node(val);
			if(isEmpty()){
				front = rear = t;
				return;
			}
			rear->next = t;
			rear = t;
		}
		void dequeue(){
			if(isEmpty()){
				return;
			}
			Node *t = front;
			front = front->next;
			if(front == NULL){
				rear = NULL;
			}
			delete t;
		}
		void display(){
			if(isEmpty()){
				cout<<"Queue is Empty!";
				return;
			}
```

```
                        Node *t = front;
                        cout<<"Queue:";
                        while(t!=NULL){
                                cout<<t->data<<" ";
                                t = t->next;
                        }
                        cout<<endl;
                }
        };
int main(){
  Queue q;
  int choice,val;
  while(true){
    cout<<"\nChoose the Queue Operation:";
    cout<<"\n1.Enqueue";
    cout<<"\n2.Dequeue";
    cout<<"\n3.Display";
    cout<<"\n4.Exit";
    cout<<"\nEnter your choice:";
    cin>>choice;
    switch(choice){
      case 1:
        cout<<"Enter value to enqueue:";
        cin>>val;
        q.enqueue(val);
        break;
      case 2:
        q.dequeue();
        break;
      case 3:
        q.display();
        break;
      case 4:
        cout<<"Exiting...";
        return 0;
      default:
        cout<<"Invalid choice!";
    }
  }
}
```

OUTPUT:





3.Implementation of circular queue using an array.

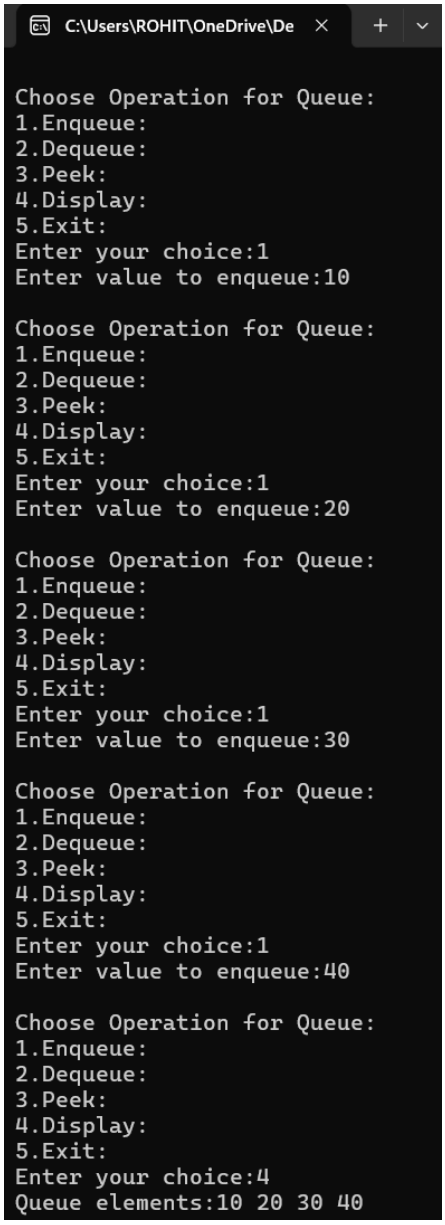
CODE:

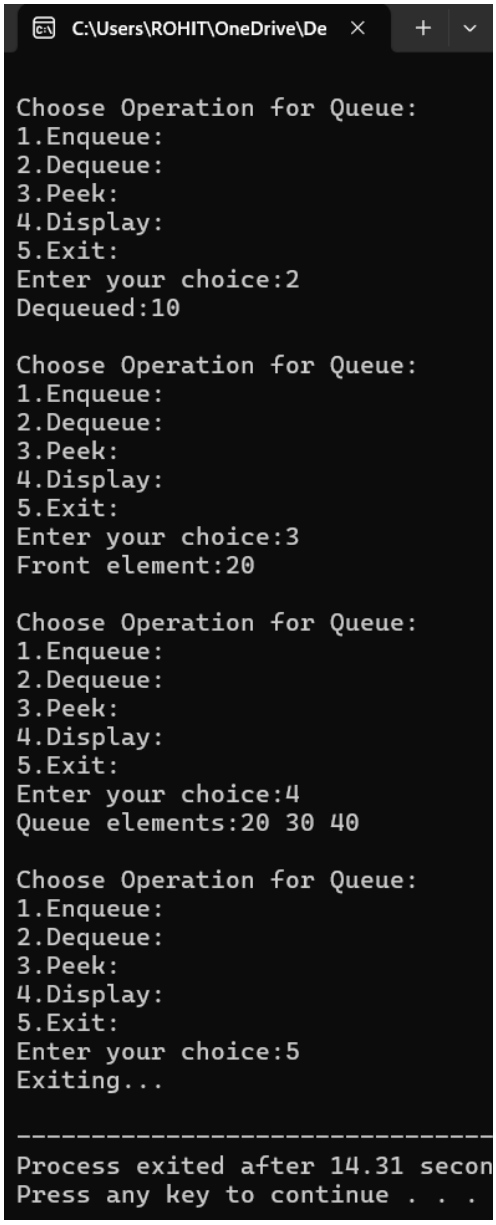
```
#include <iostream>
#define MAX_SIZE 5
#include <bits/stdc++.h>
using namespace std;
class Queue{
        public:
                int front, rear;
                int arr[MAX_SIZE];
                Queue(){
                        front = rear = 0;
                }
```

```
        bool isEmpty(){
            if(front == rear){
                return true;
            }
            return false;
        }
        bool isFull(){
            if((rear+1) % MAX_SIZE == front){
                return true;
            }
            return false;
        }
        void enqueue(int val){
            if(this->isFull()){
                cout<<"Queue Overflow!"<<endl;
                return;
            }
            rear = (rear+1) % MAX_SIZE;
            arr[rear] = val;
        }
        int dequeue(){
            if(this->isEmpty()){
                cout<<"Queue Underflow!"<<endl;
                return -1;
            }
            front = (front+1)%MAX_SIZE;
            return(arr[front]);
        }
        int peek(){
            if(this->isEmpty()){
                cout<<"Queue is empty!"<<endl;
                return -1;
            }
            return arr[(front+1)%MAX_SIZE];
        }
        void display(){
            if(this->isEmpty()){
                return;
            }
            for(int i=(front+1)%MAX_SIZE;i<rear;i=(i+1)%MAX_SIZE){
                cout<<arr[i]<<" ";
            }
            cout<<arr[rear];
        }
    };
int main(){
  Queue q;
  int choice, val;
```

```
  while(true){
    cout<<"\nChoose Operation for Queue:";
    cout<<"\n1.Enqueue:";
    cout<<"\n2.Dequeue:";
    cout<<"\n3.Peek:";
    cout<<"\n4.Display:";
    cout<<"\n5.Exit:";
    cout<<"\nEnter your choice:";
    cin>>choice;
    switch(choice){
      case 1:
        cout<<"Enter value to enqueue:";
        cin>>val;
        q.enqueue(val);
        break;
      case 2:
        val = q.dequeue();
        if(val != -1){
          cout<<"Dequeued:"<<val<<endl;
        }
        break;
      case 3:
        val = q.peek();
        if(val != -1){
          cout<<"Front element:"<<val<<endl;
        }
        break;
      case 4:
        cout<<"Queue elements:";
        q.display();
        cout<<endl;
        break;
      case 5:
        cout<<"Exiting..."<<endl;
        return 0;
      default:
        cout<<"Invalid choice!";
    }
  }
  return 0;
}
```

OUTPUT:





4.Implementation of cicular queue using doubly linked circular list.

CODE:

```
#include <iostream>
using namespace std;
class Node{
        public:
                int data;
                Node *prev, *next;
                Node(int val){
                        data = val;
                        next = prev = NULL;
                }
        };
class Queue{
```

```
private:
        Node *front, *rear;
public:
        Queue(){
                front = rear = NULL;
        }
        bool isEmpty(){
                return front==NULL;
        }
        void enqueue(int val){
                Node *t = new Node(val);
                if(isEmpty()){
                        front = rear = t;
                        front->next = front;
                        front->prev = front;
                }
                else{
                        t->prev = rear;
                        t->next = front;
                        rear->next = t;
                        front->prev = t;
                        rear =t;
                }
        }
        void dequeue(){
                if(isEmpty()){
                        cout<<"Queue is Empty!"<<endl;
                        return;
                }
                if(front == rear){
                        delete front;
                        front = rear = NULL;
                }
                else{
                        Node *t = front;
                        front = front->next;
                        front->prev = rear;
                        rear->next = front;
                        delete t;
                }
        }
        int getFront(){
                if(isEmpty()){
                        cout<<"Queue is Empty!"<<endl;
                        return -1;
                }
                return front->data;
        }
```

```
        int getRear(){
            if(isEmpty()){
                cout<<"Queue is Empty!"<<endl;
                return -1;
            }
            return rear->data;
        }
        void display(){
            if(isEmpty()){
                cout<<"Queue is Empty!"<<endl;
                return;
            }
            Node *t = front;
            cout<<"Queue:";
            do{
        cout<<t->data<<" ";
        t = t->next;
        }while(t != front);
            cout<<endl;
        }
    };
int main(){
    Queue q;
  int choice, val;
  while(true){
    cout<<"\nChoose the queue operation:";
    cout<<"\n1.Enqueue:";
    cout<<"\n2.Dequeue:";
    cout<<"\n3.Get Front:";
    cout<<"\n4.Get Rear:";
    cout<<"\n5.Display:";
    cout<<"\n6.Exit:";
    cout<<"\nEnter your choice:";
    cin>>choice;
    switch(choice){
      case 1:
        cout<<"Enter value to enqueue:";
        cin>>val;
        q.enqueue(val);
        break;
      case 2:
        q.dequeue();
        break;
      case 3:
        val = q.getFront();
        if(val != -1){
          cout<<"Front element:"<<val<<endl;
        }
```

```
        break;
      case 4:
        val = q.getRear();
        if(val != -1){
          cout<<"Rear element:"<<val<<endl;
        }
        break;
      case 5:
        q.display();
        cout<<endl;
        break;

      case 6:
        cout<<"Exiting..."<<endl;
        return 0;
      default:
        cout<<"Invalid choice!"<<endl;
        }
  }
  return 0;
}
```

OUTPUT: